Pokročilé informační systémy

Workflow systémy

doc. Ing. Radek Burget, Ph.D.
burgetr@fit.vutbr.cz

Podnikové procesy a workflow

Vývoj architektur IS, business procesy, příklady

Vývoj architektur IS

Dekáda	Vývoj
60. léta	Řada samostatných aplikací – vlastní data, UI i komunikace
70. léta	Osamostatnění dat – databázové systémy
80. léta	Osamostatnění UI – Windows API, X Window, …
90. léta	Osamostatnění řídicích procesů – **workflow systémy**

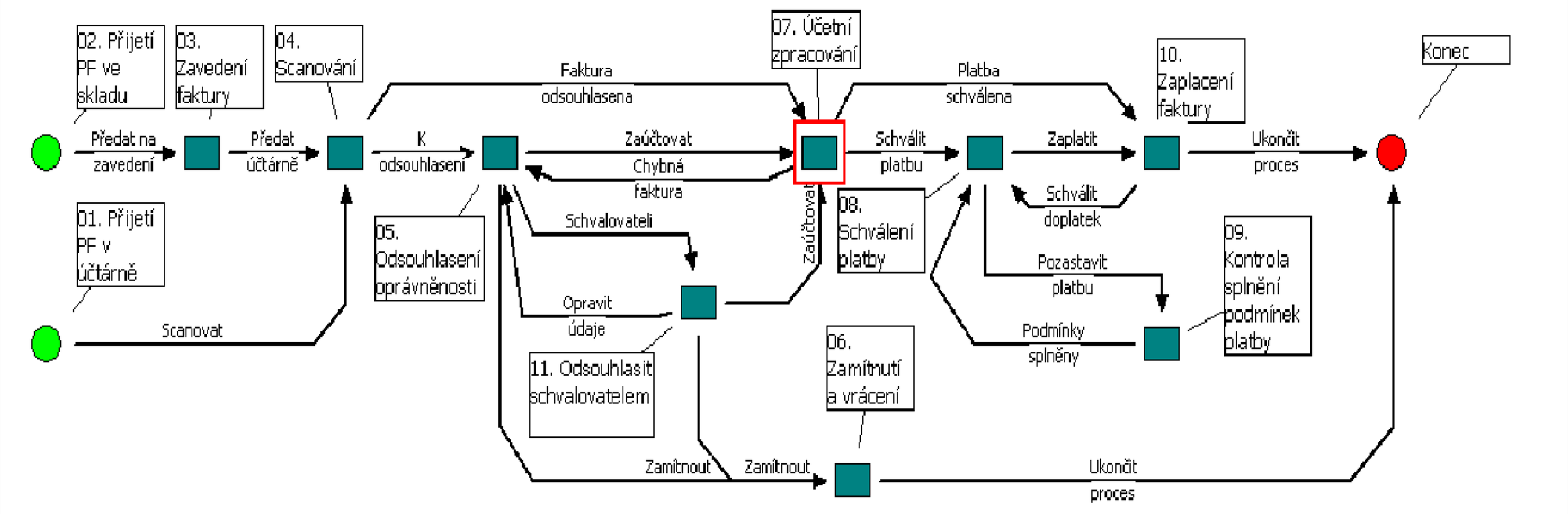
Podnikové (business) procesy

- Koordinační mechanismus napříč organizačními jednotkami
- Distribuovaný v čase a prostoru
- Integruje a koordinuje distribuované zdroje
- Poskytuje správnou informaci správnému člověku ve správný čas

CO – JAK – KDY – KDO

- Z hlediska technologie jde o:
  - Popis procesů *mimo* vlastní implementaci IS
  - Infrastrukturu schopnou zajistit vykonání popsaných procesů
  - Doplňkové funkce: monitorování, analýza, …

Příchozí faktura



Příklady procesů

- Vyřízení reklamace
  - obdržení požadavku, rozhodnutí o oprávněnosti, odpověď, …
- Vyřízení žádosti o půjčku
  - žádost, analýza rizik, schválení, sledování splátek, uzavření, …
- Zápis studentů do dalšího ročníku
  - předběžný zápis, kontrola studia, zápis, změny, …
- Výběrové řízení na zakázky
  - zadání, vyhodnocení nabídek, výběr dodavatele, realizace, …

Workflow

- **Procedurální automatizace** business procesu
- Správa sekvence pracovních aktivit a vyvolání příslušných zdrojů
- Rozdíl mezi *business procesem* a *workflow*:
  - **Business proces** – obecnější pohled, organizační perspektiva
  - **Workflow** – konkrétní popis realizace procesu, technická implementace

Od front k workflow

- Zavedli jsme **zotavitelné fronty** jako základ pro sekvenování a paralelismus transakcí
  - Fronta zaručuje: *„po dokončení akce A se někdy provede akce B"*
- **Workflow** přidává nad tento základ:
  - **Jazyk pro popis procesů** – formální definice aktivit, podmínek, větvení
  - **Role a swimlanes** – přiřazení aktivit konkrétním účastníkům
  - **Směrování** – XOR/AND/OR brány místo ručně programované logiky
  - **Monitorování a analýza** – sledování stavu instancí, výkonnostní metriky
- Popis procesu je **oddělen od implementace IS**.

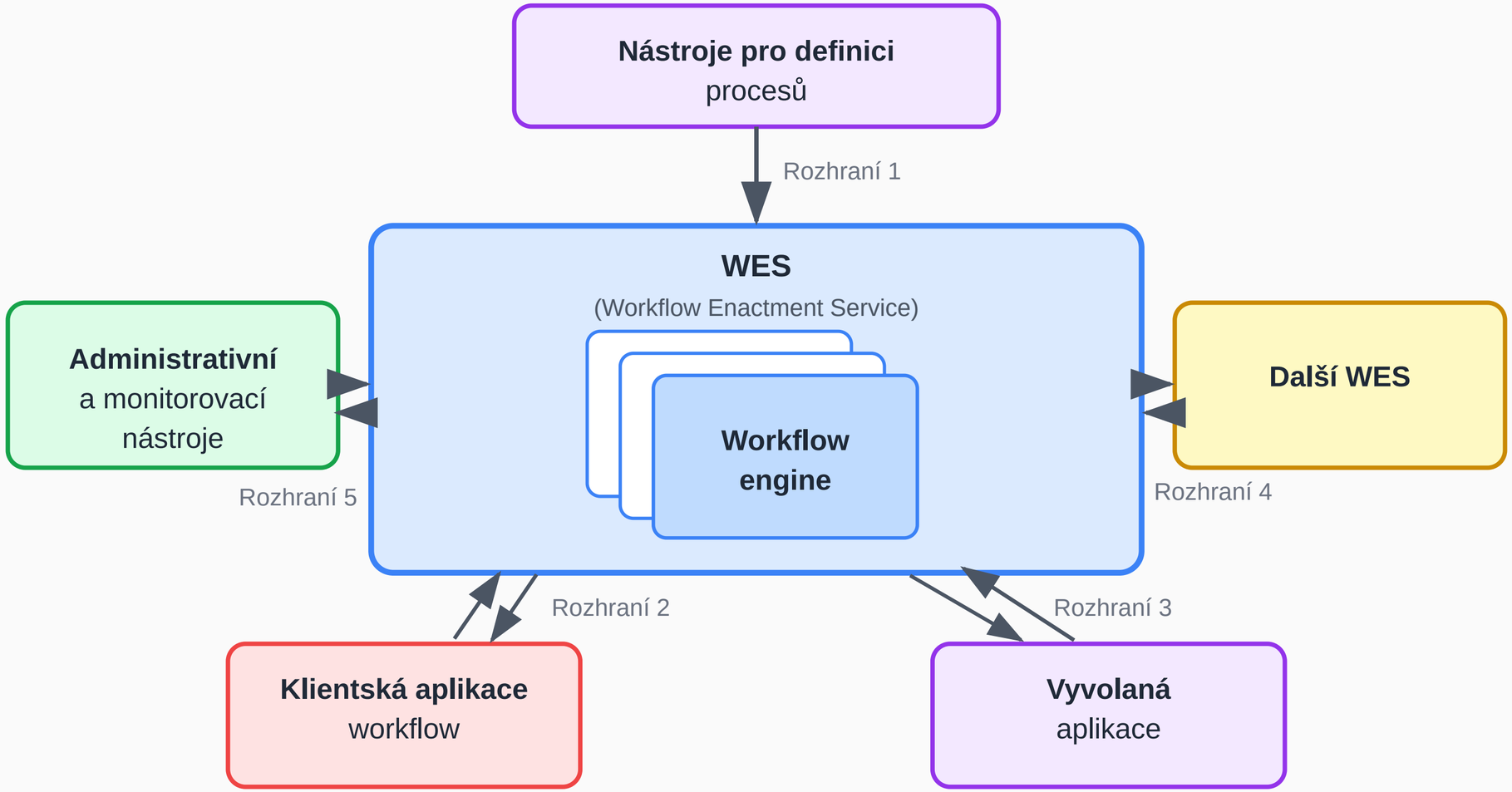
Standardy a architektura workflow

WfMC, referenční model, prvky WF systému

Standardizace – WfMC

- Velké množství SW nástrojů realizujících workflow → nutnost integrace
- **Workflow Management Coalition (WfMC)** – nevýdělečná mezinárodní organizace (1993)
- Oblasti standardizace:
  - Terminologie a referenční model
  - Spolupráce a propojení WF systémů
  - Formáty výměny definic procesů (XPDL → BPMN 2.0 XML)

Referenční model WfMC



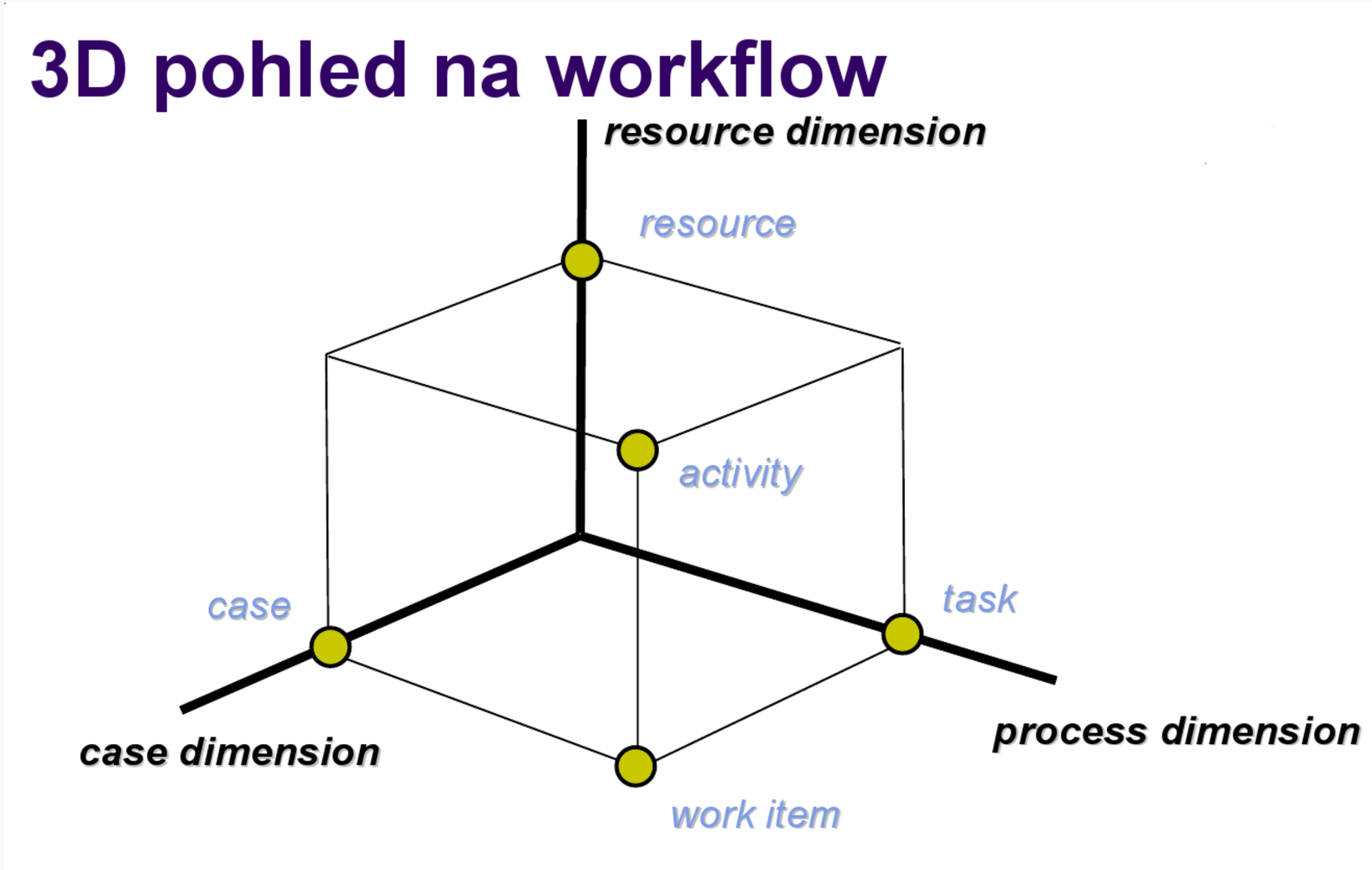
WES a Workflow engine

- **WES** (Workflow Enactment Service)
  - Zajišťuje vykonání správné činnosti pomocí správného prostředku ve správný čas
  - Složen z jednoho nebo více *workflow engines*
- **Workflow engine**
  - Interpretace definice procesu
  - Vytváří instance procesů a řídí jejich vykonávání
  - Zajišťuje přechody mezi aktivitami a vytváří pracovní položky
  - Administrace a dohled

Prvky WF systému

- **Klientské aplikace workflow** – provádějí jednotlivé úkoly, interakce uživatelů
- **Vyvolané aplikace** – spouštěné automaticky při zahájení úkolu
- **Nástroje pro definici procesů** – grafické editory (BPMN); prvky: zprávy, události, rozhodnutí
- **Nástroje pro analýzu a verifikaci**
  - Simulace: „co se stane, když…?" – ověření modelu, predikce
  - Verifikace: bude každá objednávka vyřízena? – matematické metody (**Petriho sítě**)
- **Administrace a monitorování** – sledování stavu instancí, SLA, výkonnost

3D pohled na workflow



3D pohled na workflow – dimenze

- **Případ (case)** – konkrétní řešený problém (žádost o půjčku)
  - Obvykle jej generuje externí zákazník
  - Zpracovává se prováděním úloh v určitém pořadí
- **Úloha (task)** – krok provádění procesu
  - Charakterizována podmínkami před (*precondition*) a po (*postcondition*)
- **Zdroj (resource)** – zařízení nebo osoba
  - **Role** – třída zdrojů dle schopností (např. programátoři)
  - **Organizační jednotka** – třída dle struktury (např. reklamační oddělení)
- **Pracovní položka (work item)** – úkol pro konkrétní případ
- **Činnost (activity)** – úkol + konkrétní zdroj → fronta (*worklist*)

Role ve workflow

- Práci vykonávají **kategorie pracovníků** (role)
- Jedna osoba může mít více rolí, mnoho osob má stejnou roli
- Role jsou autorizovány provádět požadavky z front spojených s činnostmi
- Přidělování požadavků: **staticky** nebo **dynamicky** (load balancing)

Data ve workflow

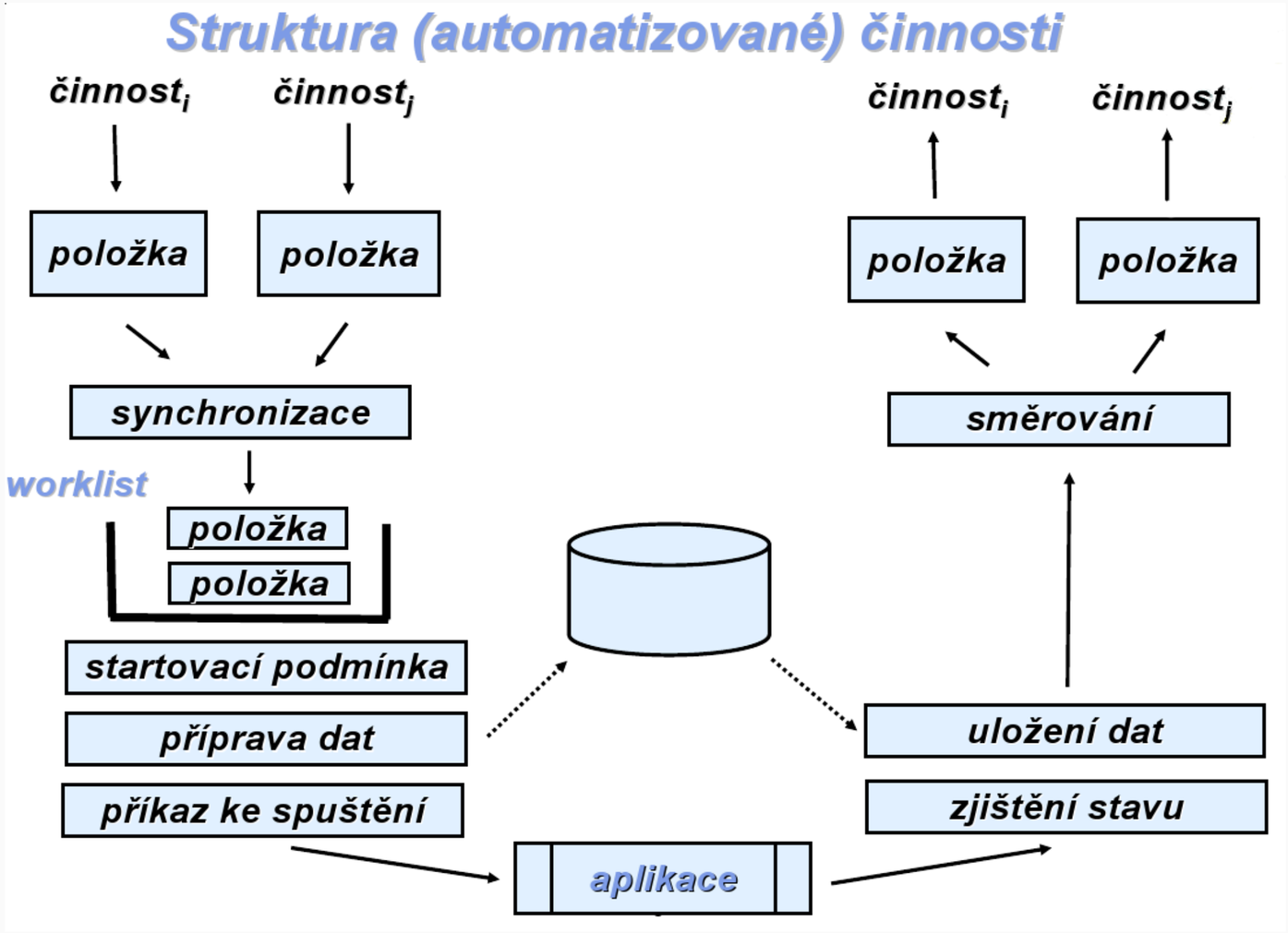
Typ dat	Popis
Řídicí data	Interní data WF systému, nedostupná externě
Věcná data	Používána pro rozhodování; dostupná i aplikacím
Aplikační data	Specifická pro aplikace; nepřístupná WF systému

- **Model organizační struktury** – role, vztahy nadřízený–podřízený
- **Definice procesu** – činnosti, přidělení rolím, rozhodovací pravidla
- **Seznam úkolů** – aktuální úkoly pro konkrétní uživatele
  - Skrytý (postupné přidělování) nebo přístupný (uživatel si volí pořadí)

Životní cyklus workflow

1. **Definice procesu** – modelování v grafickém editoru (BPMN)
2. **Nasazení** – deployment definice do WF enginu
3. **Spuštění instancí** – vytváření a řízení běžících případů
4. **Monitorování** – sledování stavu, SLA, využití zdrojů
5. **Analýza a optimalizace** – identifikace úzkých míst, úprava modelu

Struktura činnosti



Modelování business procesů

BPMN – Business Process Model and Notation

Cíle BPM

- **Formální popis** procesů probíhajících v organizaci
- **Řízení** popsaného procesu pomocí WFM systému
- **Analýza a verifikace** – zvýšení efektivity

Standardy pro modelování

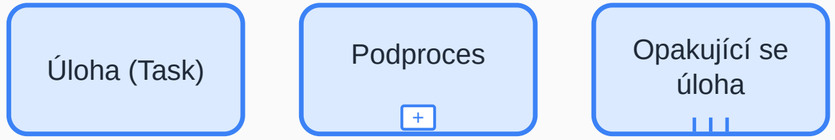
- **BPMN 2.0** (Business Process Model and Notation) – aktuální primární standard
  - Grafická notace + nativní XML serializace (BPMN XML)
  - Přímo spustitelné BPMN 2.0 enginy (Camunda, Flowable, …)
- Historické formáty (dnes legacy):
  - **XPDL** – původní deskriptivní formát WfMC, nahrazen BPMN XML
  - **BPEL** (WS-BPEL) – procedurální jazyk orientovaný na webové služby; nahrazen BPMN 2.0 enginy

Elementy BPMN

Události (Events)

Začátek | Mezilehlá | Konec | Zpráva (start) | Časovač (start)

Aktivity (Activities)



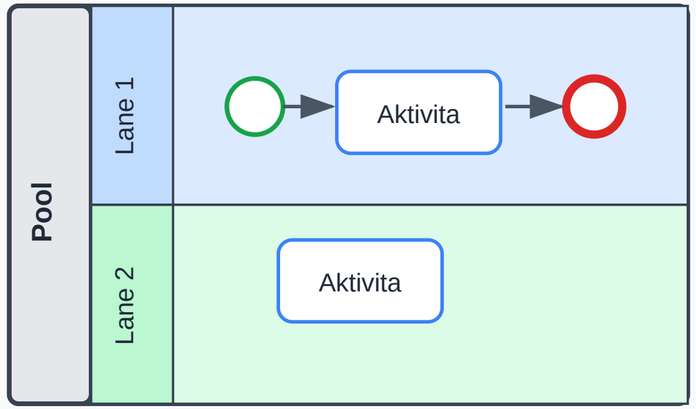
Brány (Gateways)

XOR (výlučné) | AND (paralelní) | OR (inkluzivní) | Na základě události

Spojovací objekty

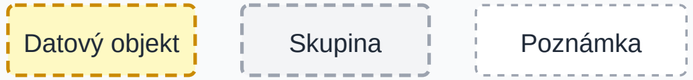
Sekvenční tok | Tok zpráv | Asociace

Swimlanes



Pool = účastník procesu
Lane = kategorie aktivit / role
Mezi pooly → tok zpráv

Artefakty



Objekty toku (Flow objects)

- **Událost (Event)**
  - Ovlivňuje tok procesu – začátek nebo konec procesu, zpráva, časovač, …
- **Aktivita (Activity)**
  - Práce, která se má vykonat – atomická úloha nebo podproces
- **Brána (Gateway)**
  - Řídí větvení a slučování toku – XOR, AND, OR
- https://camunda.com/bpmn/reference/

BPMN 2.0 – typy událostí

	Začátek (tenký okraj)	**Mezilehlá** (dvojitý okraj)	**Konec** (silný okraj)	**Boundary** (na aktivitě)	
None	Začátek	Mezilehlá	Konec	nepřerušující	přerušující
Timer	Časovač	Časovač		Časovač (boundary)	
Messa	Zpráva (start)	Zpráva	Zpráva (konec)	Zpráva (boundary)	
Error / Comp.		Chyba	Chyba (konec)	Kompenzace	Terminate (konec)

Pravidla pro boundary události

= přerušující (interrupting) – aktivita se zastaví, tok pokračuje do obslužné větve

= nepřerušující (non-interrupting) – aktivita pokračuje, spustí se paralelní tok

Boundary event je přichycen k hranici aktivity nebo podprocesu.

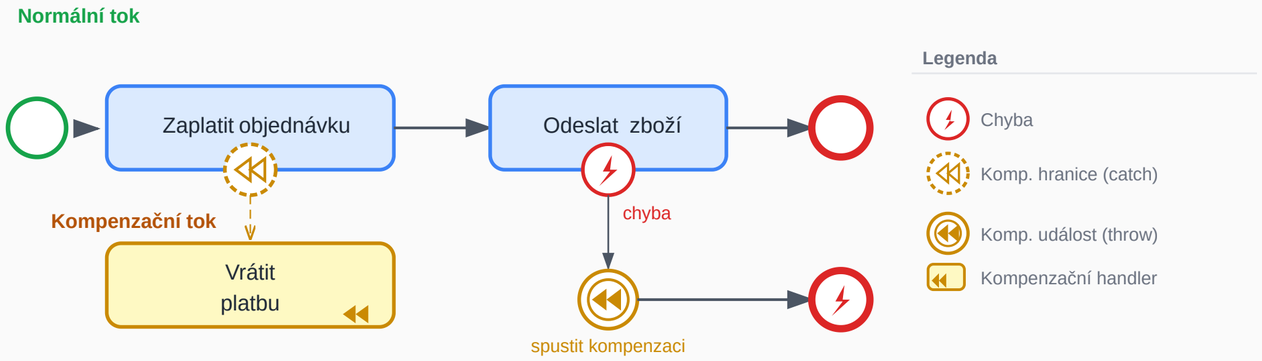
BPMN 2.0 – typy událostí (popis)

Typ	Symbol	Podtypy
Start	tenký okraj	None, Timer, Message, Signal
Intermediate	dvojitý okraj	Timer, Message, Error, Escalation
End	silný okraj	None, Message, Error, Terminate
Boundary	přichycená k aktivitě	Timer, Error, Message (přerušující / nepřerušující)

- **Přerušující** boundary event – zastaví aktivitu a přejde jinam
- **Nepřerušující** – aktivita pokračuje, spustí se paralelní tok

Kompenzace v BPMN

- BPMN 2.0 má vestavěnou podporu pro **kompenzaci** (compensation)
- **Kompenzační úloha** – aktivita, která logicky „vrátí" efekt jiné aktivity
  - Analogie s SAGA kompenzujícími transakcemi
- **Compensation event** – spouští kompenzační tok při chybě nebo explicitním požadavku
- Příklad: objednávka zaplacena → odeslání selhalo → kompenzace = vrácení platby



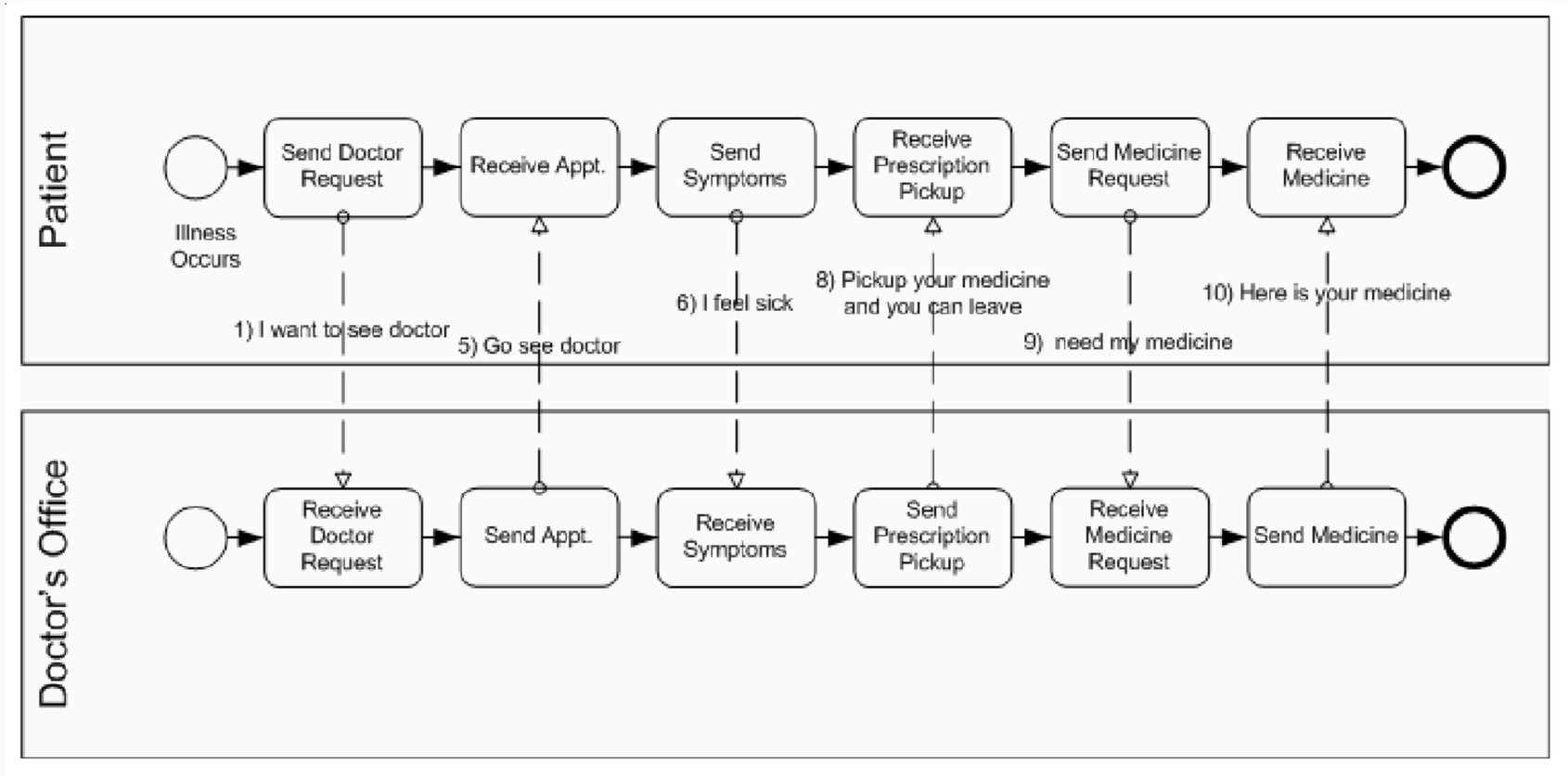
Spojovací objekty

- **Sekvenční tok** – pořadí navazujících aktivit (plná šipka)
- **Tok zpráv** – zpráva mezi dvěma účastníky procesu (přerušovaná šipka s kroužkem)
- **Asociace** – propojuje objekt s dodatečnou informací

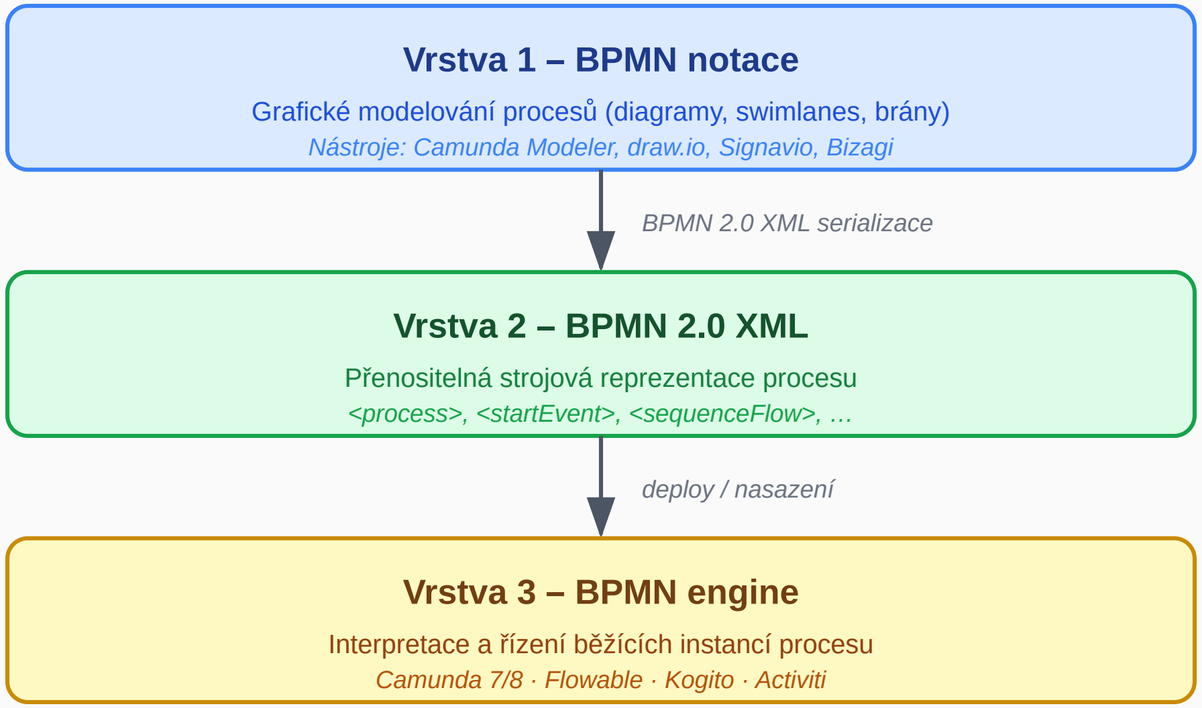
Plavecké dráhy (Swimlanes)

- **Pool**
  - Reprezentuje účastníka v procesu (organizaci, systém)
  - Mezi pooly se komunikuje tokem zpráv
- **Swimlane**
  - Kategorizuje aktivity v rámci poolu – obvykle odpovídá roli nebo oddělení

Příklad: Pool s swimlanes (pacient a lékař)



Vrstvy modelování procesů



Workflow patterns

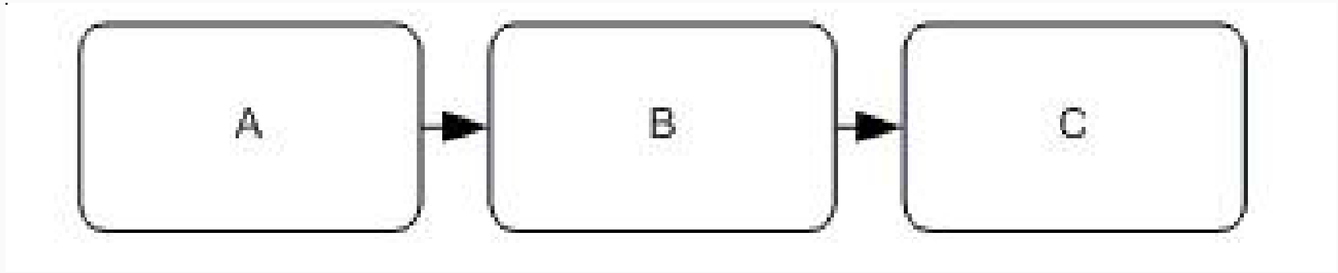
Řízení toku, směrování, flexibilita

Směrování – řízení toku

- **Split** – rozdělení toku na více větví
- **Join / Merge** – sloučení více větví do jedné
- Typy bran (gateway):
  - **XOR** – vzájemné vyloučení (právě jedna větev)
  - **AND** – paralelní zpracování (všechny větve)
  - **OR** – inkluzivní volba (jedna nebo více větví)

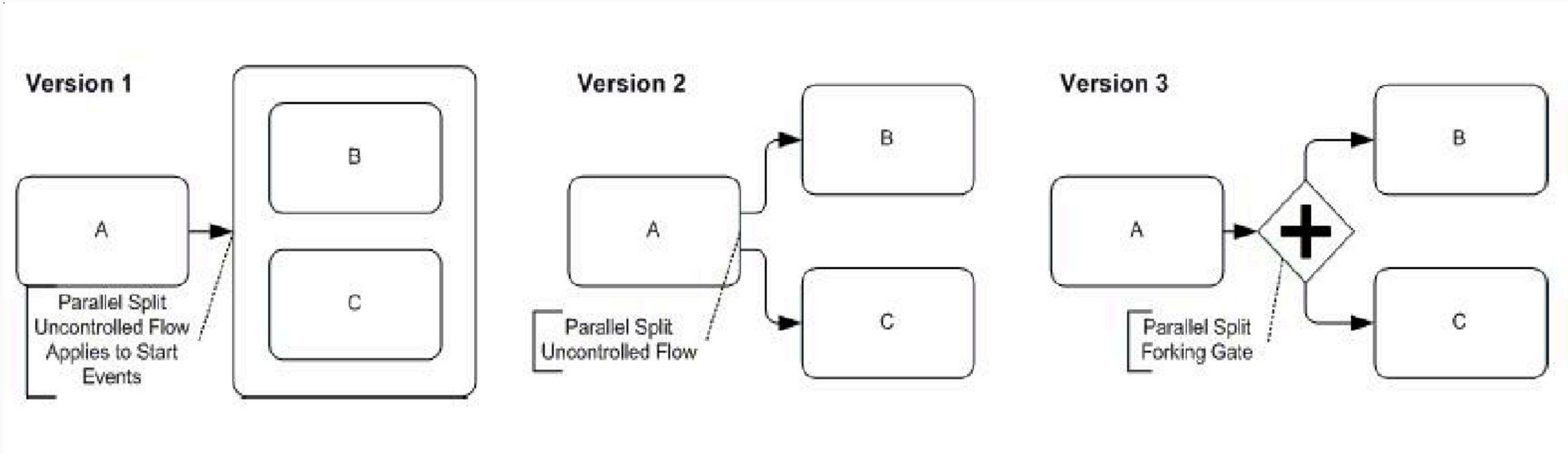
1. Sekvence

- Pracovní úkol je povolen, až je dokončeno provedení předcházejícího úkolu
- Nejzákladnější vzor – základ všech procesů



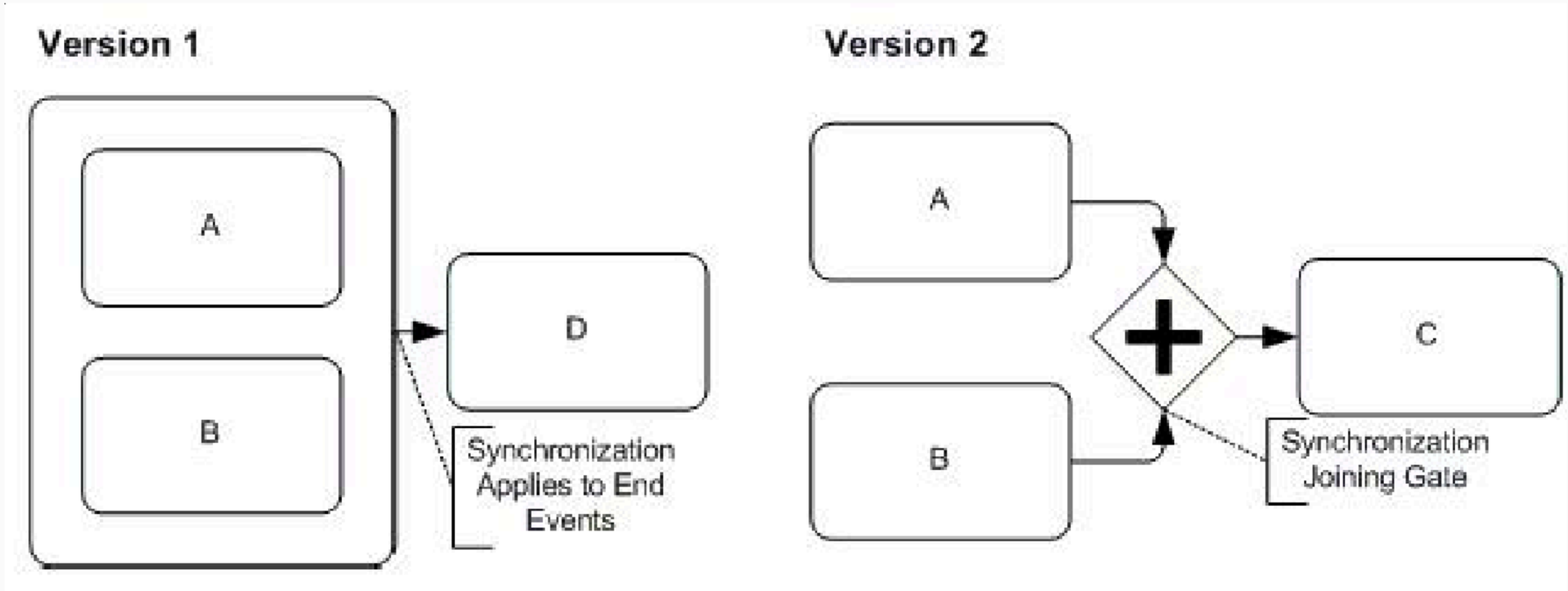
2. Parallel Split (AND-split)

- Rozděluje tok procesů do **dvou a více paralelních** vláken
- Všechny větve jsou spuštěny současně



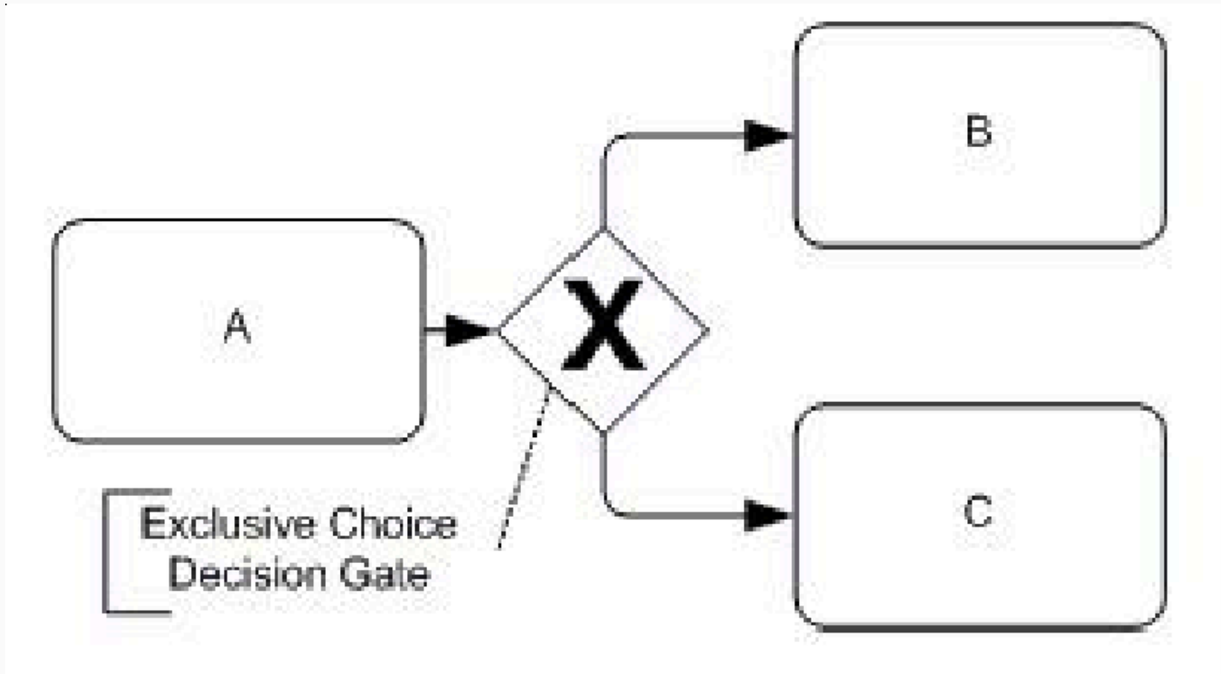
3. Synchronizace (AND-join)

- Navazující úkol začne, až jsou **dokončena všechna** předchozí vlákna
- Párový vzor k AND-split



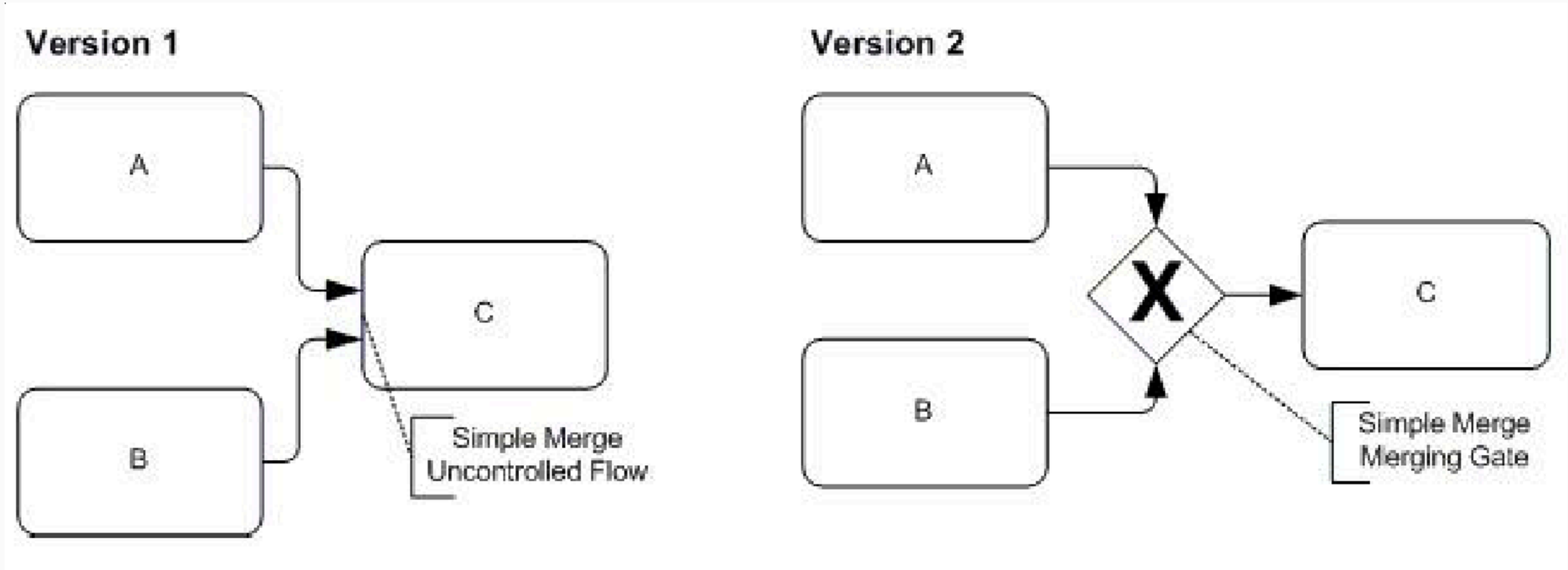
4. Výlučné rozhodnutí (XOR-split)

- Rozděluje tok na větve **vzájemně výlučné**
- Na základě podmínky (v bráně) se vstupuje do **právě jedné** z větví



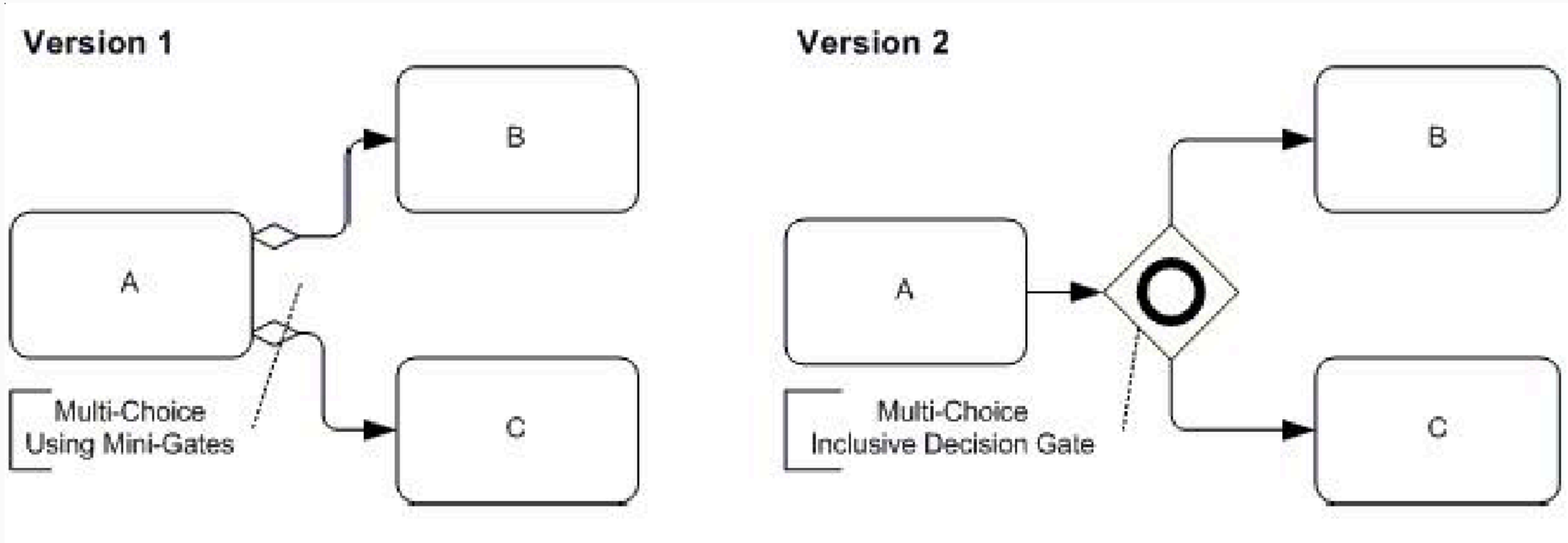
5. Jednoduché spojení (XOR-merge)

- Spojení dvou nebo více **nezávislých** větví do jedné
- Navazující aktivita začne okamžitě, jakmile **jedno** vlákno dosáhne konce
- Nemusí čekat na ostatní větve



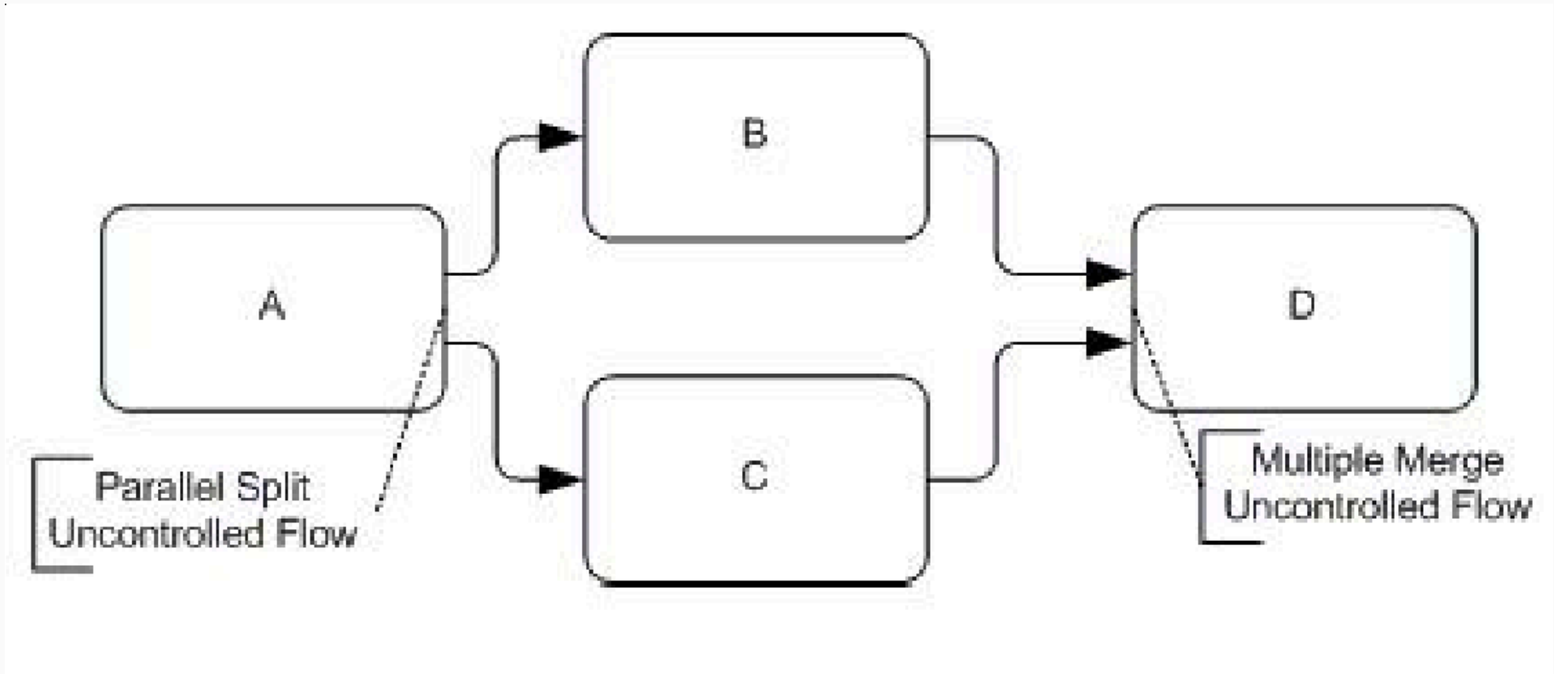
6. Vícenásobná volba (OR-split)

- Rozdělení toku do **jedné nebo více** větví
- Výběr na základě podmínek (neexkluzivní)



7. Synchronizující sloučení (OR-join)

- Čeká na ukončení **všech větví, které byly spuštěny** (ne nutně všech možných)
- Párový vzor k OR-split – „skončí vše, co začalo"



Pravidla pro přechod mezi činnostmi

- **Lhůta (deadline)** – časový limit pro provedení
- **Vstupní podmínka (pre-condition)**
  - Musí být splněna pro spuštění činnosti
  - Vyhodnocována WF systémem
- **Výstupní podmínka (post-condition)**
  - Musí být splněna pro ukončení; do té doby se činnost opakuje
- **Přechodová podmínka**
  - Umožňuje určit pořadí zpracování, např. pro mimořádné situace

Flexibilita workflow

- Podmínky pro chod organizace se mění
  - Změna legislativy, restrukturalizace, …
- Zajištění flexibility:
  - **Dopředné** – uvažujeme všechny možné situace předem
  - **Zpětné** – **změna workflow za běhu** (dynamická evoluce)

Dynamická změna workflow

- Pomocí **evolution patterns** – základní typy změn
  - Vždy definováno, zda lze změnu provést a jak
- Převod existujících instancí:
  - *Concurrent to completion* – stávající instance doběhnou původním způsobem
  - *Migrace na finální schéma* – jen za určitých podmínek
  - *Migrace na ad-hoc schéma*
- **Verifikace výsledku** – bude WF stále dělat to, co má?
  - Analýza cest, dosažitelnost stavů
  - **Petriho sítě**

Concurrent to completion

- Existující instance pokračují podle **původního schématu** až do svého přirozeného konce
- Nové instance se spouštějí podle **nového schématu**
- Nejjednodušší a nejbezpečnější přístup
- Nevýhoda: po dobu přechodu existují v systému instance **různých verzí**
  - Komplikuje monitoring a reporting

Migrace na finální schéma

- Existující instance jsou **převedeny přímo na nové schéma**
- Podmínky proveditelnosti migrace:
  - Instance musí být v **kompatibilním stavu** – nesmí být uprostřed aktivity, která zanikla nebo se zásadně změnila
  - Stav instance musí být jednoznačně namapovatelný na stav v novém schématu
- Zachovává konzistenci – všechny instance běží podle **jedné verze** schématu

Migrace na ad-hoc schéma

- Existující instance jsou převedeny na **dočasné přechodové schéma**
  - „Most" mezi starým a novým schématem
- Použití: přímá migrace na finální schéma není možná (nekompatibilní stavy)
- Ad-hoc schéma definuje, jak instance „uprostřed" starého procesu dojde do stavu kompatibilního s novým schématem
- Nejflexibilnější, ale nejsložitější na **implementaci a verifikaci**

Technologie a nástroje

BPMN 2.0 enginy, moderní přístupy, process mining

BPMN 2.0 – strojová reprezentace

```
<process processType="Private" isExecutable="true"
         id="com.sample.HelloWorld" name="Hello World">

  <!– nodes –>
  <startEvent id="_1" name="StartProcess"/>
  <scriptTask id="_2" name="Hello">
    <script>System.out.println("Hello World");</script>
  </scriptTask>
  <endEvent id="_3" name="EndProcess">
    <terminateEventDefinition/>
  </endEvent>
```

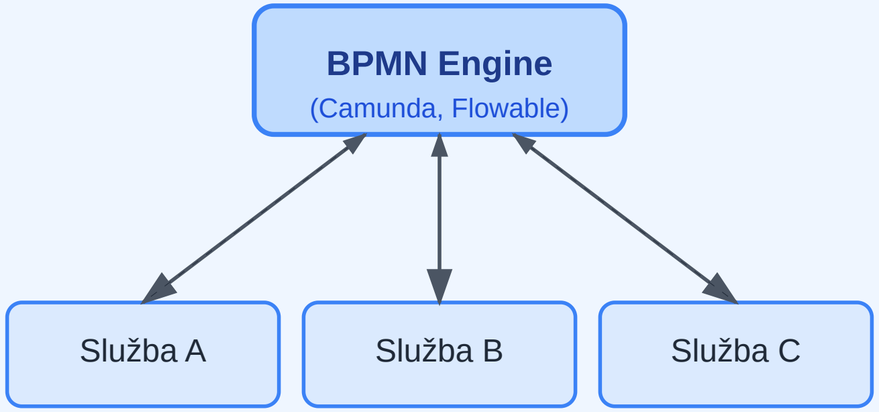
Moderní BPMN enginy

Engine	Typ	Poznámka
Camunda 7	open-source, Java	Nejrozšířenější BPMN engine; REST API, embedded i standalone
Camunda 8 / Zeebe	cloud-native	Distribuovaný engine; SaaS i self-hosted; škálovatelný
Flowable	open-source, Java	Odvozen z Activiti; lehčí, Spring Boot integrace
Activiti	open-source, Java	Základ ekosystému; spravován Alfresco
Kogito (Red Hat)	cloud-native	Nástupce jBPM; Quarkus/Kubernetes; BPMN + DMN
Bonitasoft	open-source low-code	Grafické studio, citizen developer přístup

Orchestrace vs. choreografie

Orchestrace

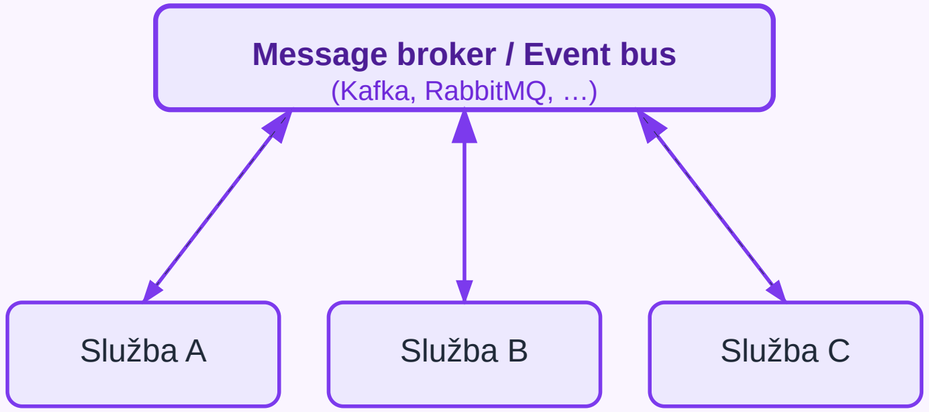
(centrální BPMN engine)



- ✔ Centrální viditelnost stavu
- ✔ Snadné ladění a monitoring
- ✔ Přímá implementace BPMN
- ✘ Engine = single point of failure
- ✘ Možný bottleneck při škálování

Choreografie

(event-driven, bez centrálního koordinátora)



- ✔ Přirozeně škálovatelná
- ✔ Žádný single point of failure
- ✔ Volné vazby mezi službami
- ✘ Distribuovaný stav – těžší sledování
- ✘ Složitější ladění a testování

Orchestrace vs. choreografie – srovnání

	Orchestrace	**Choreografie**
Koordinátor	Centrální BPMN engine	Žádný; každá služba reaguje na události
Viditelnost procesu	Engine zná celý stav	Distribuovaný stav, těžší sledování
SAGA implementace	Engine orchestruje $T_1 \ldots T_n$ a kompenzace	Každá služba naslouchá a vydává události
Škálovatelnost	Engine = možný bottleneck	Přirozeně škálovatelná
Ladění	Snadné (centrální log)	Složitější (distribuované sledování)
Příklad	Camunda + REST volání	Apache Kafka + event-driven služby

Moderní distribuované workflow

- **Temporal** (temporal.io) – workflow jako kód (Java, Python, Go, TypeScript)
  - Centrální engine (v Go)
  - Proces popsaný kódem (Python, Java, …)
  - Automatický retry, timeouty, zotavení bez ztráty stavu
  - Vhodné pro: orchestraci mikroslužeb, dlouhotrvající procesy
- **Cloudové managed služby:**
  - **AWS Step Functions** – vizuální stavový stroj, serverless, integrace Lambda/SQS
  - **Azure Logic Apps** – low-code, rozsáhlý konektor ekosystém
  - **Google Cloud Workflows** – YAML/JSON definice, integrace GCP služeb
  - Společné: serverless model, platba za přechody; nevýhoda: vendor lock-in

Process mining

- **Process mining** – automatické *objevování procesů* z event logů
  - Event log = záznamy z IS nebo workflow enginu:
    `[caseID, aktivita, čas]`
- Klíčové úlohy:
  - **Discovery** – rekonstrukce modelu procesu z logů
  - **Conformance checking** – porovnání skutečného průběhu s BPMN
  - **Enhancement** – doplnění modelu o výkonnostní metriky
- Nástroje:
  - **ProM** – akademická platforma (TU/e, van der Aalst)
  - **Celonis** – komerční
  - **Disco** / **Bupar** (Python) – lehčí nástroje

Literatura

- W.M.P. van der Aalst: *Process Mining: Data Science in Action* (2. vyd., Springer, 2016)
- Camunda: docs.camunda.org – referenční BPMN engine dokumentace
- Temporal: docs.temporal.io – distribuované workflow jako kód
- workflowpatterns.com – katalog workflow patterns (van der Aalst et al.)

A to je vše!

Dotazy?